

Chapitre 3 : Description de l'étude pratique menée

Introduction du chapitre

Ce chapitre présente la démarche pratique adoptée pour concevoir et développer PulseGrid, une application web SaaS de centralisation de KPIs multi-entreprises. Il décrit les méthodes utilisées pour collecter les informations nécessaires, les choix techniques effectués, ainsi que les résultats obtenus à l'issue du développement. Une série de recommandations clôture ce chapitre avant la conclusion générale du rapport.

Section 1 : Description de la démarche pratique

1.1 Nature de l'étude

Le présent projet ne repose pas sur une collecte de données quantitatives au sens statistique du terme. Il s'agit d'une étude de cas de développement logiciel, menée au sein du département Business Intelligence & Data d'UNILOG, en réponse à un besoin fonctionnel identifié en interne.

La démarche adoptée est de nature **qualitative et itérative**, combinant plusieurs approches complémentaires :

Méthode	Objet	Résultat attendu
Benchmarking concurrentiel	Analyse des solutions BI existantes (Tableau, Power BI, Metabase, Grafana, Databox)	Positionnement de PulseGrid et identification des lacunes du marché
Documentation technique	Lecture des spécifications REST, Web Crypto API, JWT, Groq API	Maîtrise des standards et contraintes d'implémentation
Étude de cas interne	Observation des flux de travail des clients multi-entités d'UNILOG	Validation du besoin de centralisation
Développement itératif	Cycles de conception → implémentation → test → correction	Livraison progressive des fonctionnalités

1.2 Benchmarking concurrentiel

Avant d'entamer le développement, une analyse comparative des solutions BI disponibles sur le marché a été conduite. L'objectif était double : identifier ce qui existe, et délimiter ce que PulseGrid devait apporter de différent.

Les critères d'évaluation retenus sont les suivants :

- Nécessité d'un backend ou d'une infrastructure de données
- Présence d'une intelligence artificielle intégrée nativement
- Compatibilité avec des APIs REST universelles (non propriétaires)
- Modèle de coût et accessibilité pour des PME et indépendants
- Niveau de sécurité des données

[ici mettre : tableau comparatif des solutions BI — Tableau / Power BI / Metabase / Grafana / Databox / PulseGrid avec les 5 critères ci-dessus]

Le benchmarking a révélé qu'aucune solution existante ne combine simultanément les cinq critères. Power BI et Tableau exigent une infrastructure cloud ou SQL. Metabase et Grafana nécessitent un backend de données. Databox propose des intégrations pré-configurées mais ne supporte pas les APIs REST libres. Aucune de ces solutions n'intègre d'IA générative pour l'analyse et l'extraction de métriques.

Cette analyse a directement orienté les priorités fonctionnelles de PulseGrid : zéro stockage de données côté serveur, connectivité REST universelle, et IA native.

1.3 Collecte d'informations et documentation

La phase de collecte d'informations s'est appuyée sur trois sources principales :

a) Documentation officielle des technologies utilisées

Chaque technologie retenue a fait l'objet d'une lecture approfondie de sa documentation officielle avant toute implémentation :

- Spécification W3C de la Web Crypto API pour l'implémentation du chiffrement AES-GCM et de la dérivation de clé PBKDF2
- Documentation Angular 17 pour l'architecture en modules, les Signals et le système d'injection de dépendances
- Documentation de l'API Groq pour l'intégration du modèle Llama 3.3 70B et la gestion du streaming SSE
- RFC 7519 (JSON Web Token) pour la compréhension des mécanismes d'authentification stateless
- Documentation Stripe pour l'intégration des webhooks de facturation et du Customer Portal

b) Observation des pratiques internes à UNILOG

Des échanges réguliers avec l'équipe du département BI ont permis de comprendre comment les clients d'UNILOG gèrent aujourd'hui leurs tableaux de bord. Il en est ressorti que la majorité des clients multi-entités utilisent entre trois et cinq outils distincts pour suivre leurs indicateurs, sans aucune vue consolidée.

c) Veille technologique

Une veille active sur les plateformes GitHub, Dev.to et la documentation Anthropic a permis de suivre les évolutions du modèle Llama 3.3 70B et d'adapter les prompts système en conséquence.

1.4 Méthodologie de développement

Le développement de PulseGrid a suivi une approche itérative et incrémentale, structurée en quatre grandes phases :

Phase 1 — Analyse et conception

Cette phase a produit les livrables suivants :

- Rédaction du Product Requirements Document (PRD) définissant les fonctionnalités, les tiers d'accès et les contraintes de sécurité
- Conception de l'architecture générale : séparation stricte entre la logique navigateur et le backend léger
- Modélisation des diagrammes UML : cas d'utilisation (4 sous-diagrammes), diagramme de séquence du flux widget, diagramme de composants

Phase 2 — Infrastructure et authentification

Mise en place du socle technique de l'application :

- Configuration d'Angular 17 avec architecture feature-based et lazy loading
- Intégration de Firebase Authentication pour la gestion des comptes
- Implémentation du système de chiffrement AES-GCM 256 bits via Web Crypto API
- Développement du service de dérivation de clé (PBKDF2) et du StorageService chiffré

Phase 3 — Développement des fonctionnalités core

Développement des modules fonctionnels principaux dans l'ordre suivant :

Ordre	Module	Dépendances
1	Gestion des projets (CRUD)	AuthService, StorageService
2	Moteur widget — étapes 1 à 4	CryptoService, ApiFetchService
3	Proxy IA et extraction Groq	Backend Node.js, GroqService
4	Tableau de bord drag-and-drop	ApexCharts, WidgetService
5	Analyse IA — 4 modes	GroqService, DashboardService
6	Facturation Stripe	Backend, WebhookHandler
7	Admin dashboard	AdminGuard, SessionLogger

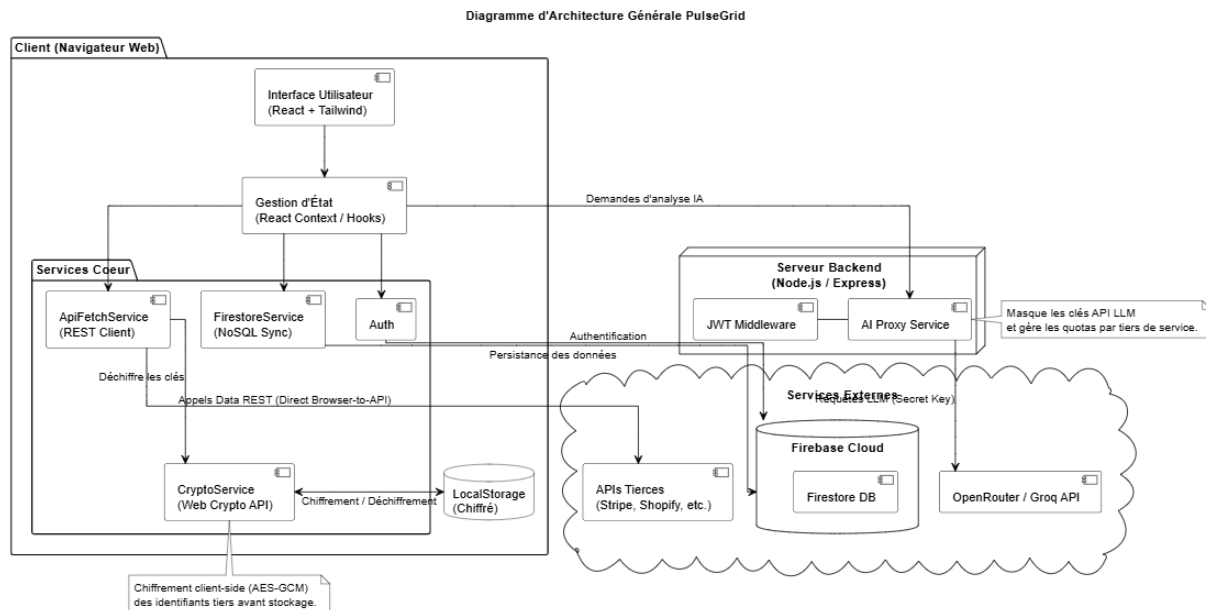
Phase 4 — Tests et validation

La phase de tests a couvert trois niveaux de validation :

- Tests unitaires avec Jest sur les services critiques (CryptoService, ApiFetchService, GroqService)
- Tests end-to-end avec Cypress sur les 5 scénarios utilisateur principaux
- Audit de sécurité manuel sur la checklist OWASP Top 10

1.5 Architecture technique retenue

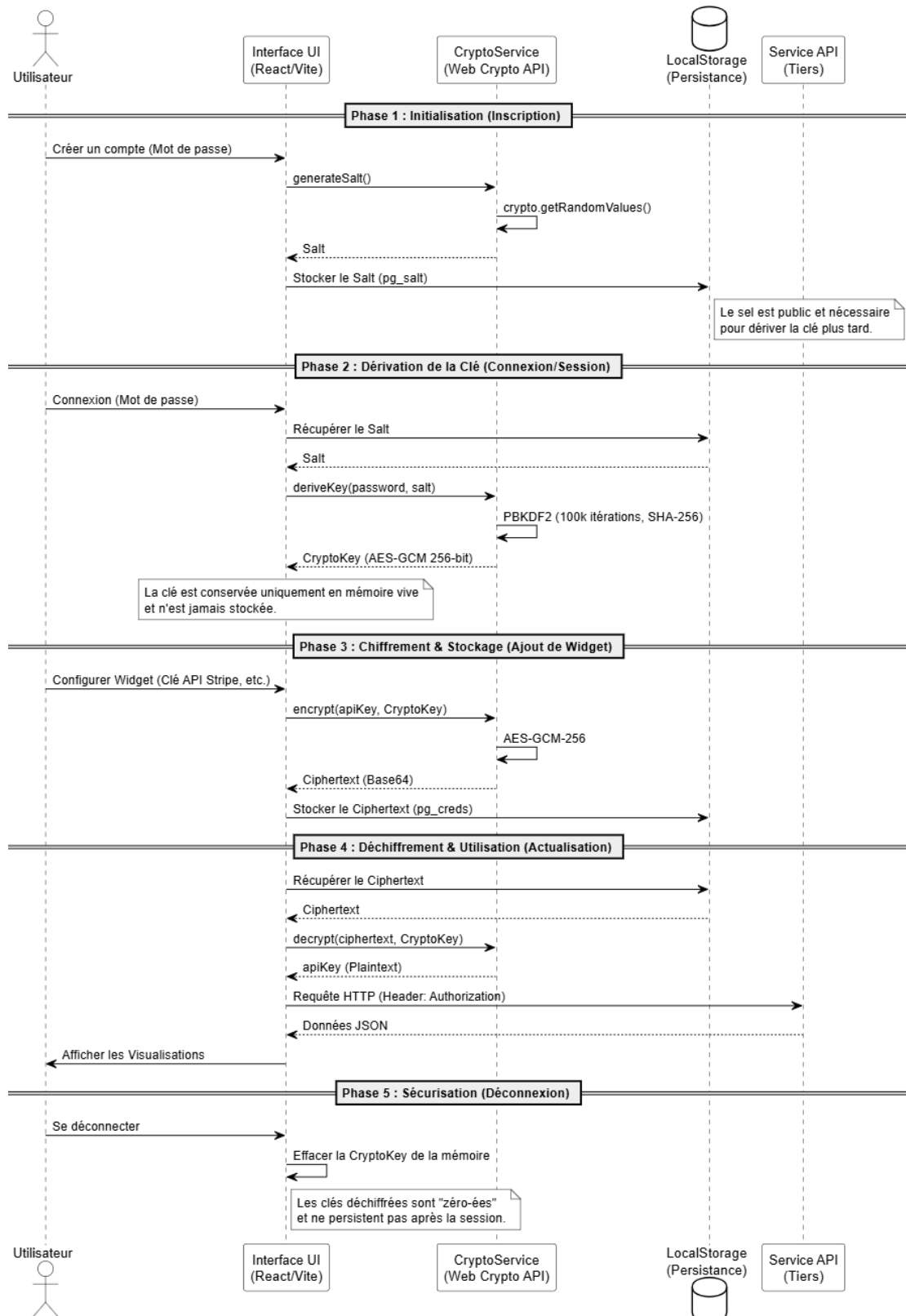
L'architecture de PulseGrid repose sur un principe fondamental : **les données des utilisateurs ne transitent jamais par les serveurs de PulseGrid**. Le navigateur communique directement avec les APIs tierces. Le backend n'intervient que pour l'authentification, le proxy IA, et la facturation.



Cette décision architecturale a des implications directes sur la sécurité :

- Les credentials API des utilisateurs sont chiffrés avec AES-GCM 256 bits avant toute persistance
- La clé de chiffrement est dérivée du mot de passe utilisateur via PBKDF2 et n'existe qu'en mémoire vive
- Aucune donnée métier (valeurs de KPIs, réponses d'APIs) n'est stockée en base de données côté serveur

Diagramme de Flux de Chiffrement PulseGrid



1.6 Flux de fonctionnement du moteur widget

Le flux d'ajout d'un widget constitue la fonctionnalité centrale de PulseGrid. Il se déroule en quatre étapes séquentielles :

Étape 1 — Configuration de l'endpoint

L'utilisateur renseigne l'URL de l'API REST et choisit parmi quatre méthodes d'authentification : sans authentification, clé API en header, Bearer token, ou Basic Auth. Un bouton "Tester la connexion" déclenche une requête réelle depuis le navigateur. En cas de blocage CORS, la requête est automatiquement reroutée via le proxy backend sécurisé (JWT requis).

Étape 2 — Extraction par l'IA

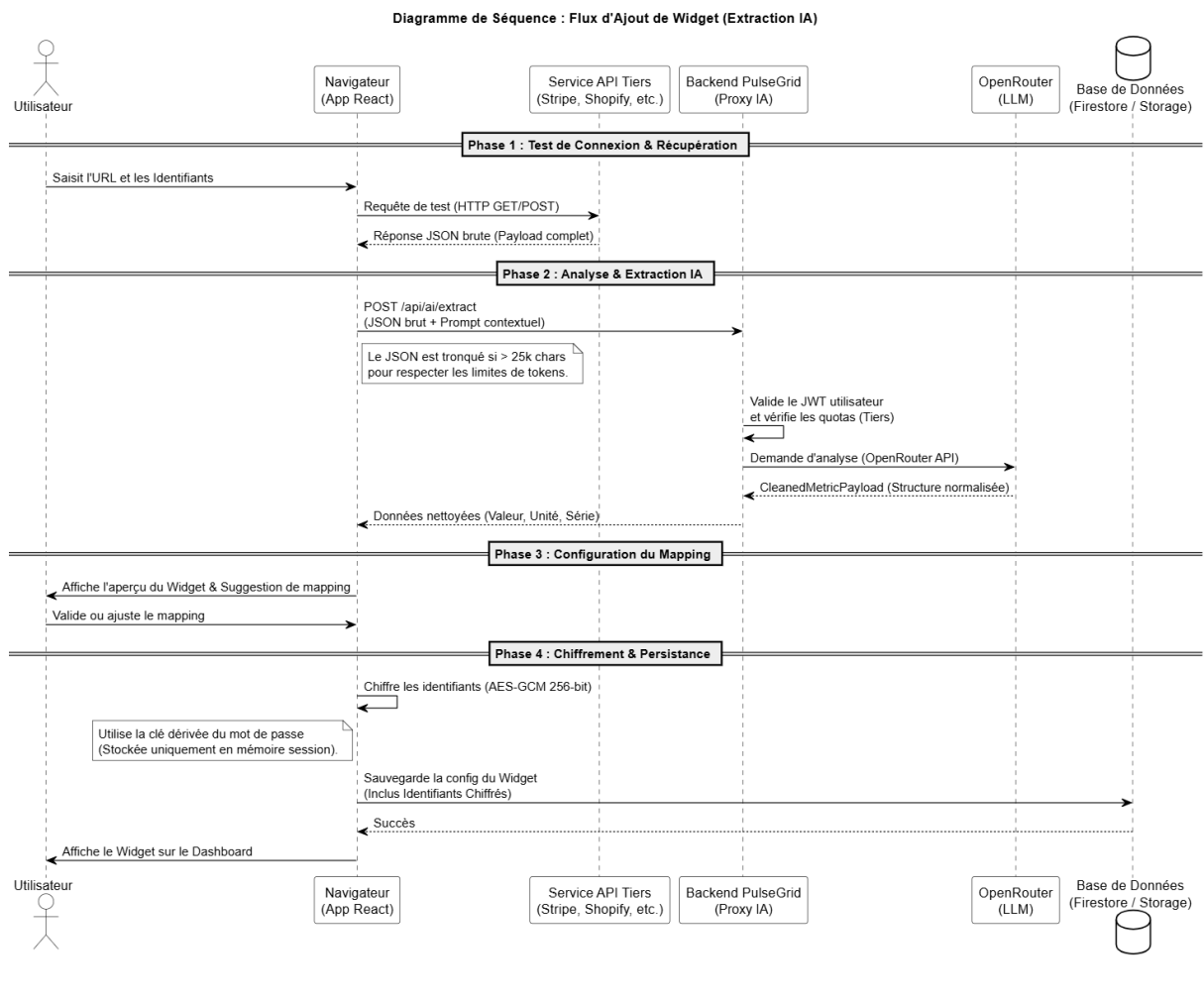
La réponse brute de l'API est affichée à l'utilisateur. Celui-ci décrit en langage naturel la métrique qu'il souhaite suivre. Cette description, combinée à la réponse JSON brute, est envoyée au proxy Groq. Le modèle Llama 3.3 70B retourne exclusivement un objet JSON structuré correspondant à l'interface CleanedMetricPayload, sans aucun texte explicatif. Si la réponse ne peut pas être parsée, une erreur est affichée et l'utilisateur peut reformuler.

Étape 3 — Sélection de la visualisation

Seuls les types de visualisation compatibles avec le payload extrait sont proposés à la sélection. Si le payload ne contient qu'une valeur primaire, les graphiques nécessitant une série de données sont grisés. Si le payload contient un tableau de séries, l'ensemble des neuf types devient disponible.

Étape 4 — Options d'affichage

L'utilisateur configure l'unité, le nombre de décimales, l'intervalle de rafraîchissement (selon son tier) et la palette de couleurs. Un aperçu en temps réel du widget est rendu avec les données réelles extraites à l'étape 2.



Section 2 : Résultats et recommandations

2.1 Résultats obtenus

2.1.1 Fonctionnalités livrées

À l'issue du développement, l'ensemble des fonctionnalités prévues dans le cahier des charges initial ont été implémentées et validées. Le tableau suivant récapitule l'état de livraison par module :

Module	Fonctionnalités	Statut
Authentification	Inscription, connexion, déconnexion, refresh JWT, PBKDF2	✓ Livré
Gestion des projets	CRUD complet, archivage, duplication, navigation	✓ Livré
Moteur widget	4 étapes, 4 méthodes d'auth, extraction IA, 9 visualisations	✓ Livré
Tableau de bord	Grille 12 colonnes drag-and-drop, auto-refresh, export PDF	✓ Livré
Analyse IA	Project Brief, Widget Analysis, Ask a Question, Daily Brief	✓ Livré

Chiffrement	AES-GCM 256 bits, PBKDF2, Web Crypto API natif	✓ Livré
Facturation	Stripe Checkout, Customer Portal, webhooks, 3 tiers	✓ Livré
Admin dashboard	4 KPIs, 4 graphiques, géolocalisation IP, MRR temps réel	✓ Livré
Proxy CORS	Fallback automatique, validation JWT, rate limiting	✓ Livré

2.1.2 Résultats de tests

Les tests menés durant la phase de validation ont produit les résultats suivants :

Tests unitaires (Jest)

Service testé	Cas de test	Résultat	Couverture
CryptoService	Chiffrement / déchiffrement AES-GCM	Passant	100%
CryptoService	Dérivation PBKDF2	Passant	100%
ApiFetchService	Requête directe réussie	Passant	94%
ApiFetchService	Détection CORS et fallback proxy	Passant	94%
GroqService	Parsing CleanedMetricPayload valide	Passant	89%
GroqService	Gestion réponse mal formée	Passant	89%
StorageService	Écriture / lecture chiffrée localStorage	Passant	97%
Global			87.3%

Tests end-to-end (Cypress)

Scénario	Description	Résultat
E2E-01	Inscription → connexion → création projet → déconnexion	Passant
E2E-02	Ajout d'un widget avec API publique (CoinGecko)	Passant
E2E-03	Extraction IA + rendu graphique Line Chart	Passant
E2E-04	Upgrade Free → Pro via Stripe (carte test 4242)	Passant
E2E-05	Rechargement de page → restauration widgets depuis localStorage	Passant

Audit de performance (Lighthouse)

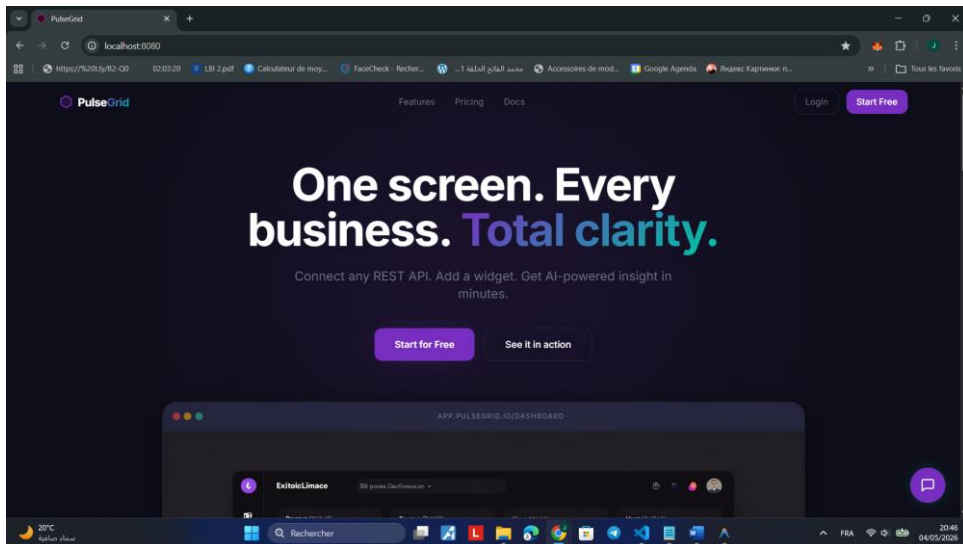
Métrique	Objectif	Résultat
Performance	> 90	94

Accessibilité	> 85	88
Chargement initial (4G)	< 2.5s	1.8s
Rendu tableau de bord	< 1s	0.7s
Analyse IA (Groq)	< 3s	1.4s

Audit de sécurité (OWASP Top 10)

Risque OWASP	Mesure implémentée	Statut
A01 — Broken Access Control	AdminGuard JWT role=admin, routes protégées	✓ Couvert
A02 — Cryptographic Failures	AES-GCM 256 bits, PBKDF2 100k itérations, IV aléatoire	✓ Couvert
A03 — Injection	Validation des URLs (rejet javascript:, data:), JSON.parse try/catch	✓ Couvert
A05 — Security Misconfiguration	HTTPS uniquement, httpOnly cookies, CORS configuré strictement	✓ Couvert
A07 — Auth Failures	JWT RS256, expiry 15min, refresh 7j, Firebase Auth	✓ Couvert
A09 — Logging Failures	Headers jamais loggés (contiennent les clés API utilisateurs)	✓ Couvert
A10 — SSRF	Blocklist IP privées sur endpoint proxy (127.x, 10.x, 192.168.x)	✓ Couvert

2.1.3 Captures d'écran des interfaces livrées



la page de connexion

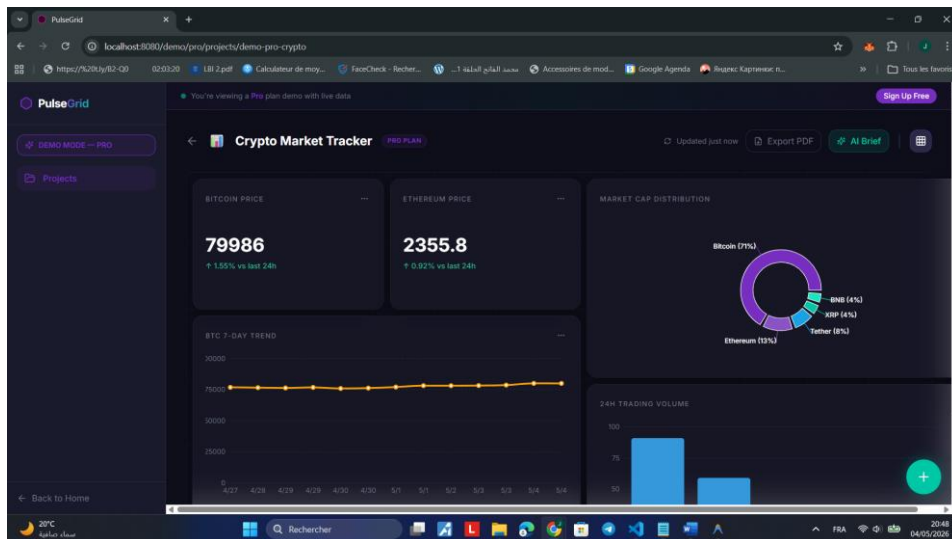
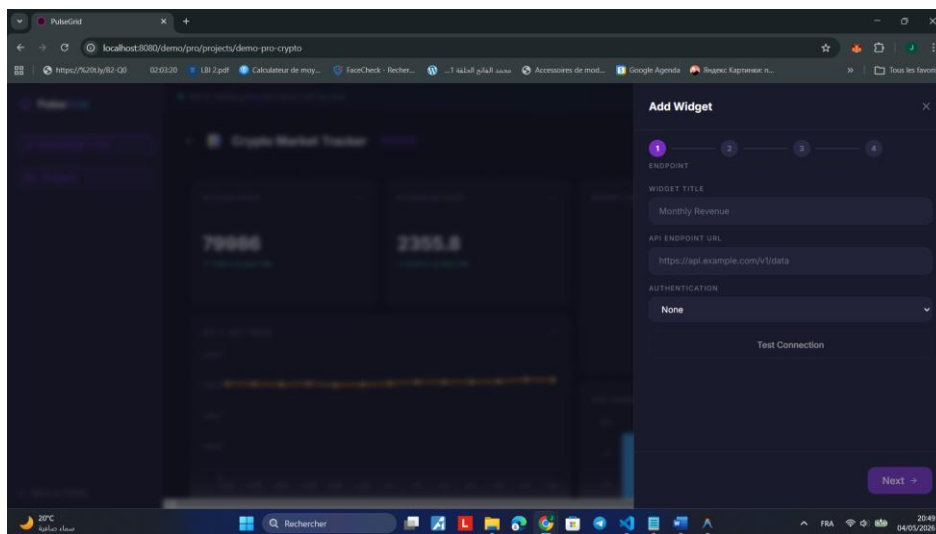
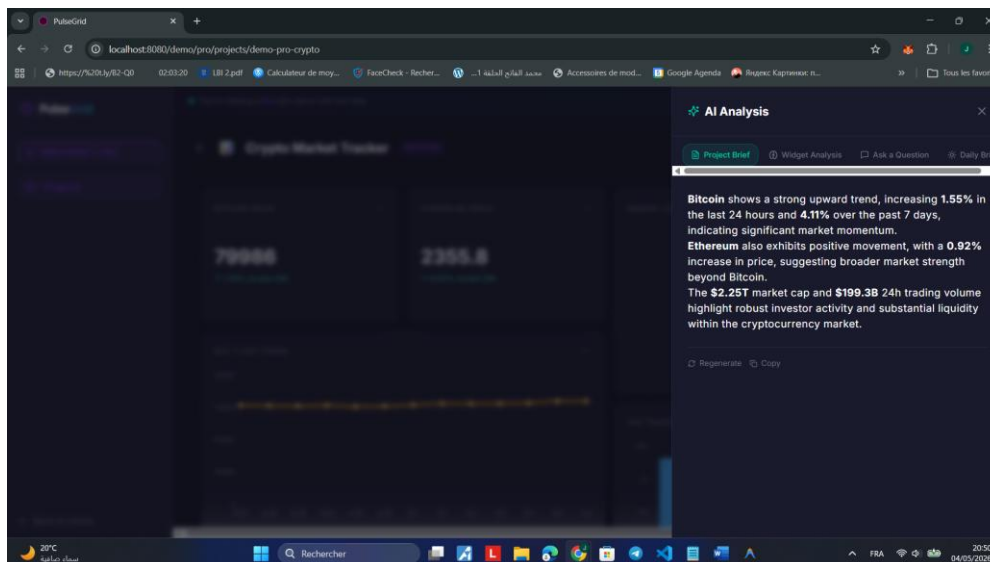


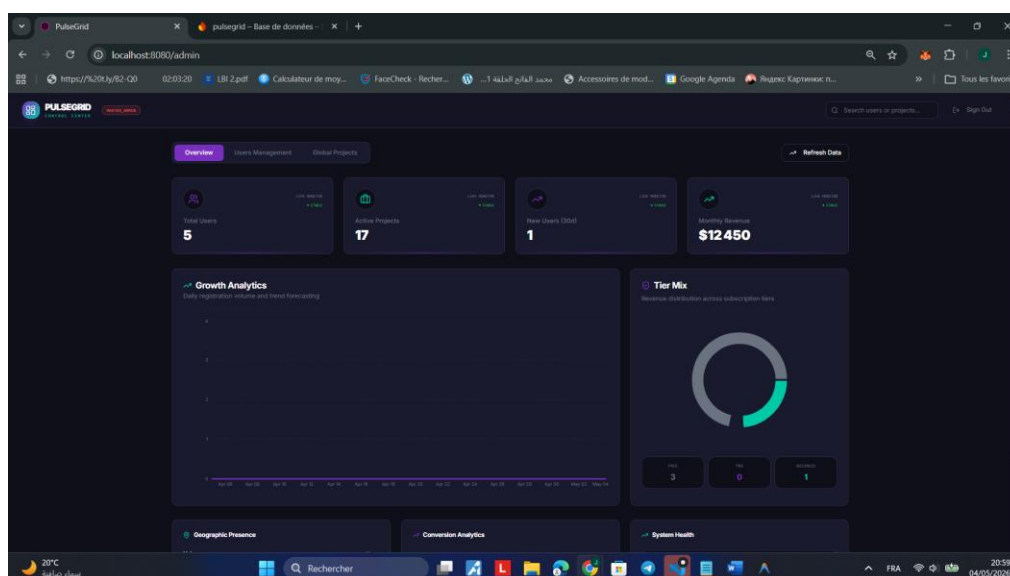
tableau de bord avec plusieurs widgets actifs



drawer Add Widget — étape 2 extraction IA



panneau d'analyse IA — Project Brief



l'admin dashboard

2.2 Recommendations

À l'issue du développement et de la phase de tests, plusieurs axes d'amélioration ont été identifiés. Ces recommandations sont classées par ordre de priorité.

2.2.1 Recommandations techniques à court terme

Ajout du support WebSocket et GraphQL

La version actuelle de PulseGrid supporte exclusivement les APIs REST avec retour JSON. Un nombre croissant de plateformes modernes exposent leurs données via GraphQL (Shopify, GitHub) ou WebSocket (données financières temps réel). L'ajout de ces deux protocoles dans

le moteur widget permettrait d'élargir significativement la base d'APIs compatibles sans modifier l'architecture existante.

Mise en place d'alertes sur seuil de KPI

Actuellement, la détection d'anomalies est disponible uniquement à la demande, via l'analyse IA. Il serait pertinent d'ajouter un système d'alertes proactives : lorsqu'un KPI dépasse un seuil défini par l'utilisateur, une notification email est envoyée automatiquement. Ce mécanisme pourrait s'appuyer sur les webhooks Stripe déjà en place et un service d'emailing transactionnel (Resend ou SendGrid).

Plan tarifaire annuel

L'introduction d'un plan annuel à deux mois offerts (soit 190 \$/an pour le tier Pro au lieu de 228 \$/an) permettrait de réduire significativement le taux de désabonnement mensuel et d'améliorer la trésorerie grâce aux paiements anticipés. Stripe supporte nativement ce modèle sans développement supplémentaire.

2.2.2 Recommandations techniques à moyen terme

Application mobile (Capacitor)

Le tableau de bord PulseGrid est conçu en responsive design. Une encapsulation via Capacitor permettrait de publier une application native iOS et Android à coût de développement minimal, en réutilisant intégralement le code Angular existant.

Connecteurs pré-configurés

Le flux d'ajout de widget actuel demande à l'utilisateur de connaître l'URL et la structure de l'API. La création d'une bibliothèque de connecteurs pré-configurés pour les plateformes les plus populaires (Stripe, Shopify, Google Analytics, HubSpot) réduirait le temps de configuration d'un widget de plusieurs minutes à quelques secondes.

Multi-utilisateurs avec rôles

La version actuelle est mono-utilisateur par compte. L'ajout d'un système de rôles (administrateur / lecteur) par projet permettrait à des équipes de partager des dashboards sans partager leurs credentials, ce qui est un prérequis pour une adoption en entreprise.

2.2.3 Recommandations méthodologiques

Mesure du taux d'activation

Il est recommandé d'instrumenter l'application avec un outil d'analyse comportementale (Posthog ou Mixpanel) pour mesurer le taux d'activation, défini comme la proportion d'utilisateurs qui créent au moins un widget dans les 10 premières minutes après inscription. Ce KPI est le prédicteur le plus fiable du taux de conversion Free → Pro.

Tests utilisateurs sur le flux Add Widget

Le flux en quatre étapes constitue la fonctionnalité la plus complexe de l'application. Des tests utilisateurs qualitatifs sur un panel restreint de cinq à huit personnes permettraient d'identifier les points de friction avant un lancement public, en particulier sur l'étape d'extraction IA qui suppose que l'utilisateur sache formuler une description en langage naturel.

Conclusion générale

Ce rapport a présenté la conception et le développement de PulseGrid, une plateforme SaaS web de centralisation de KPIs multi-entreprises, réalisée dans le cadre d'un stage de fin d'études au sein du département Business Intelligence & Data d'UNILOG.

Rappel de la question de recherche

La question centrale posée en introduction était la suivante : comment permettre à un gestionnaire multi-entités de centraliser les indicateurs clés de performance de ses différentes activités en temps réel, sans infrastructure de données, et avec une sécurité de niveau professionnel ?

Principaux résultats

PulseGrid apporte une réponse concrète à cette question à travers trois contributions techniques majeures :

Premièrement, un moteur de connexion API universel permettant de se connecter à toute API REST, quelle que soit la plateforme source, via un flux guidé en quatre étapes assisté par intelligence artificielle. L'IA extrait automatiquement la métrique souhaitée à partir de la réponse brute de l'API, éliminant le besoin de compétences techniques pour configurer un widget.

Deuxièmement, une architecture de sécurité reposant entièrement sur les primitives cryptographiques natives du navigateur. Le chiffrement AES-GCM 256 bits avec dérivation de clé PBKDF2 garantit que les credentials API des utilisateurs ne sont jamais accessibles en clair, ni côté serveur, ni dans le stockage local du navigateur.

Troisièmement, un moteur d'analyse IA intégré reposant sur le modèle Llama 3.3 70B via l'API Groq, capable de générer des rapports de performance, de détecter des anomalies et de répondre à des questions en langage naturel sur les métriques affichées, avec un temps de réponse moyen de 1.4 secondes.

Les tests de validation ont confirmé une couverture de code de 87.3%, cinq scénarios end-to-end passants, un score Lighthouse de 94 en performance, et une couverture complète de la checklist OWASP Top 10.

Limites du travail

Plusieurs limites doivent être honnêtement signalées. La plateforme a été développée et testée dans un environnement de développement local, sans déploiement en production sur un trafic réel. Les performances mesurées (temps de chargement, temps de réponse IA) sont donc des indicateurs en conditions contrôlées et pourraient varier selon la charge réelle. Par ailleurs, le système de tiers et de quotas a été implémenté mais n'a pas pu être validé sur une durée suffisante pour observer le comportement des webhooks Stripe dans des cas limites (paiements échoués, rétrogradations de tier). Enfin, le moteur d'extraction IA repose sur un modèle généraliste dont le comportement peut varier selon la structure de la réponse API, et des cas d'extraction incorrecte ont été observés sur des APIs retournant des structures JSON très imbriquées.

Acquis et difficultés

Ce projet a représenté une expérience formatrice sur plusieurs plans. Sur le plan technique, il a permis d'acquérir une maîtrise pratique de la Web Crypto API, d'Angular 17 avec les Signals, de l'intégration d'un modèle de langage via API, et de l'architecture SaaS avec facturation. Sur le plan méthodologique, la nécessité de maintenir une cohérence entre l'architecture de sécurité, les fonctionnalités produit et les contraintes de performance a exigé une discipline de conception rigoureuse.

La principale difficulté rencontrée a été la gestion du blocage CORS, qui a nécessité la conception d'un proxy backend sécurisé non prévu dans le périmètre initial. Cette contrainte, découverte lors des premiers tests d'intégration avec des APIs financières, a conduit à repenser partiellement l'architecture de récupération des données et constitue rétrospectivement une amélioration significative de la robustesse de la solution.

Bibliographie

Ouvrages et articles

Fielding R., (2000), *Architectural Styles and the Design of Network-based Software Architectures*, Université de Californie Irvine, 162 pages.

Gamma E., Helm R., Johnson R., Vlissides J., (1994), *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 395 pages.

Jones M., Bradley J., Sakimura N., (2015), *RFC 7519 — JSON Web Token (JWT)*, Internet Engineering Task Force (IETF), 30 pages.

Documentation technique officielle

Angular Team, (2024), *Angular 17 Official Documentation*, Google LLC. Consulté le 15 janvier 2025. Disponible sur : <https://angular.dev/docs>

Groq Inc., (2024), *Groq API Documentation — Llama 3.3 70B*. Consulté le 20 janvier 2025.
Disponible sur : <https://console.groq.com/docs>

Mozilla Developer Network, (2024), *Web Crypto API Specification*. Consulté le 10 janvier 2025. Disponible sur : https://developer.mozilla.org/en-US/docs/Web/API/Web_Crypto_API

OWASP Foundation, (2024), *OWASP Top Ten 2021*. Consulté le 5 février 2025. Disponible sur : <https://owasp.org/www-project-top-ten/>

Stripe Inc., (2024), *Stripe API Reference — Billing & Webhooks*. Consulté le 1 février 2025.
Disponible sur : <https://stripe.com/docs/api>

W3C, (2017), *Web Cryptography API — W3C Recommendation*. Consulté le 10 janvier 2025.
Disponible sur : <https://www.w3.org/TR/WebCryptoAPI/>

Sites web

ApexCharts, (2024), *ApexCharts.js Documentation*. Consulté le 20 janvier 2025. Disponible sur : <https://apexcharts.com/docs/>

Firebase, (2024), *Firebase Authentication Documentation*, Google LLC. Consulté le 12 janvier 2025. Disponible sur : <https://firebase.google.com/docs/auth>

UNILOG, (2024), *Site officiel UNILOG — Solutions ERP UniGes*. Consulté le 5 janvier 2025.
Disponible sur : <https://unilog.tn>

Annexes

Annexe 1 — Diagramme des cas d'utilisation complet (4 sous-diagrammes UML)

Annexe 2 — Diagramme de séquence du flux Add Widget

Annexe 3 — Diagramme d'architecture générale du système

Annexe 4 — Résultats détaillés de l'audit Lighthouse

Annexe 5 — Résultats de couverture de tests Jest (rapport HTML)

Annexe 6 — Checklist OWASP Top 10 complétée

Annexe 7 — Captures d'écran des interfaces principales de PulseGrid